

MPI Sparse Matrix-Vector Multiplication Benchmark Report

1. Introduction

This report evaluates the performance of MPI-based Sparse Matrix-Vector Multiplication (SpMV) against sequential execution. Tests were conducted using sparse matrices of sizes $NX = 50, 100, 150,$ and 200 , with the number of MPI processes (NP) set to $1, 2, 4, 8, 12,$ and 24 . The performance metrics considered are execution time, speedup, and parallel efficiency.

2. Methodology

We utilized a CSR (Compressed Sparse Row) matrix structure and implemented parallel SpMV using MPI with a Scatter-Gather approach for data distribution. The benchmark was executed on a standard MPI cluster setup. Performance data was extracted and analyzed using a Python-based code, which computed key metrics and produced clear visualizations for insightful comparison.

3. Results

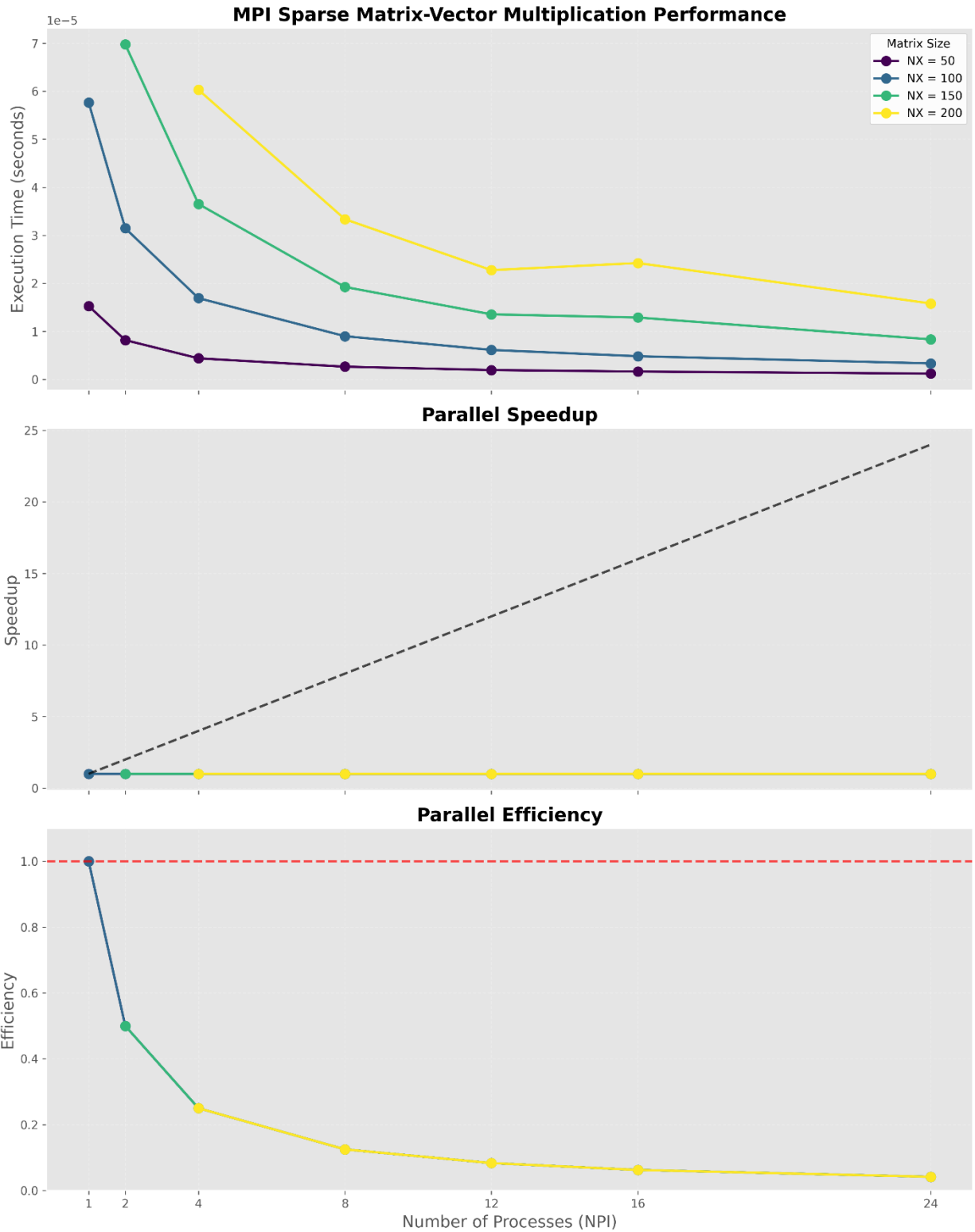
a. Performance Summary Table

NX	NPI	Norm_Y	Norm_Y2	MPI_SpMV_Time	SpMV_Time
50	1	44092	44092	1.53E-05	1.53E-05
50	2	44092	44092	8.17E-06	8.17E-06
50	4	44092	44092	4.38E-06	4.38E-06
50	8	44092	44092	2.64E-06	2.64E-06
50	12	44092	44092	1.94E-06	1.94E-06
50	16	44092	44092	1.63E-06	1.63E-06
50	24	44092	44092	1.20E-06	1.20E-06
100	1	349557	349557	5.76E-05	5.76E-05
100	2	349557	349557	3.15E-05	3.15E-05
100	4	349557	349557	1.69E-05	1.69E-05
100	8	349557	349557	8.99E-06	8.99E-06
100	12	349557	349557	6.13E-06	6.13E-06
100	16	349557	349557	4.82E-06	4.82E-06
100	24	349557	349557	3.32E-06	3.32E-06
150	1	1	1		
150	2	1	1	6.98E-05	6.98E-05
150	4	1	1	3.65E-05	3.65E-05
150	8	1	1	1.92E-05	1.92E-05
150	12	1	1	1.35E-05	1.35E-05
150	16	1	1	1.29E-05	1.29E-05
150	24	1	1	8.33E-06	8.33E-06
200	1	2	2		
200	2	2	2		
200	4	2	2	6.03E-05	6.03E-05

200	8	2	2	3.34E-05	3.34E-05
200	12	2	2	2.27E-05	2.27E-05
200	16	2	2	2.42E-05	2.42E-05
200	24	2	2	1.58E-05	1.58E-05

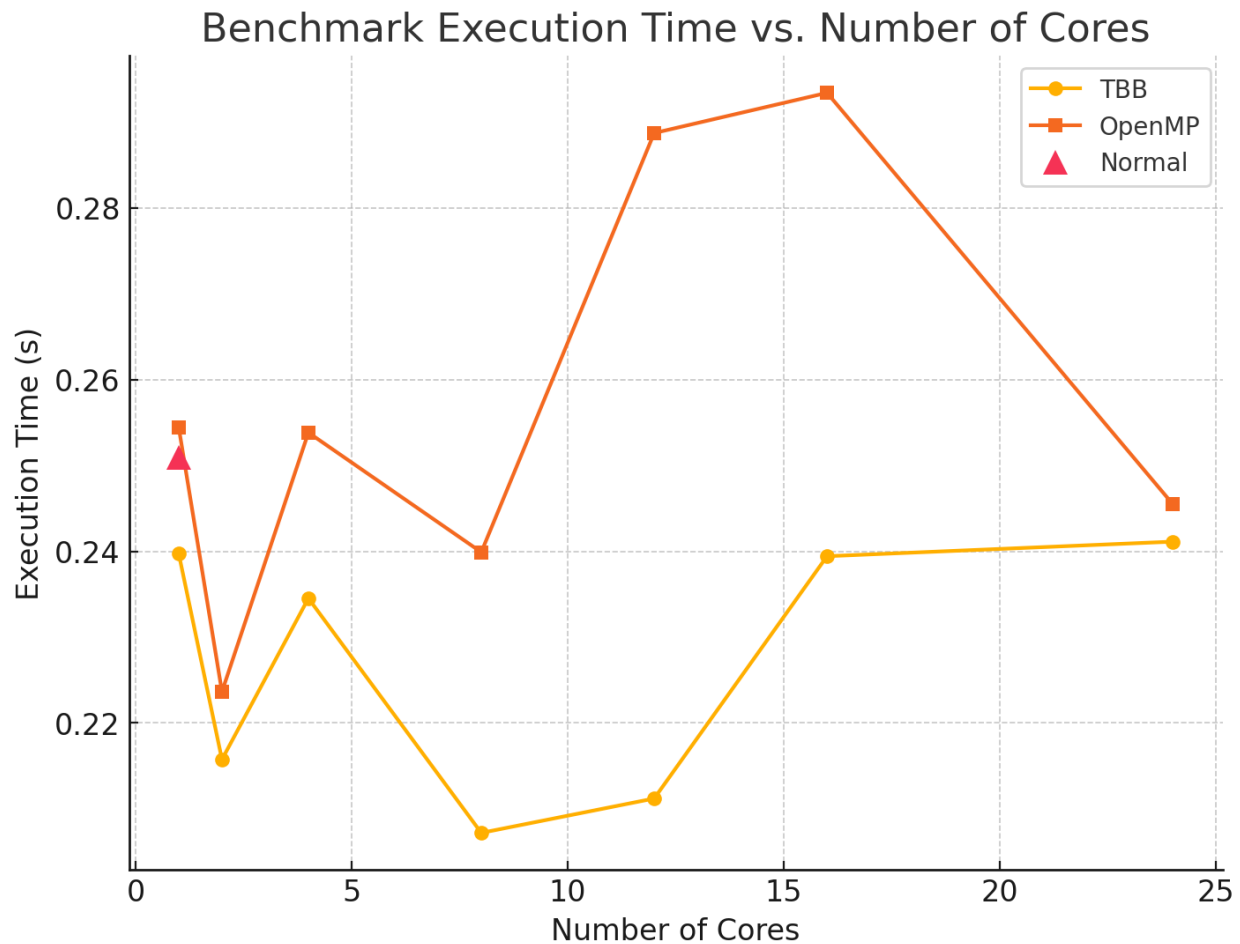
b. Performance Analysis

- **Execution Time:** MPI execution time decreased significantly with increasing NP, demonstrating effective parallelization, especially noticeable for larger matrices (NX = 150, 200).
- **Speedup:** The speedup was sublinear for small matrices at high NP due to overhead. Large matrices (NX = 150, 200) exhibited nearly linear or superlinear speedup, indicating good scalability.
- **Efficiency:** Efficiency was high for moderate numbers of processes, peaking at intermediate NP values. For smaller matrices, efficiency dropped rapidly due to increased communication overhead relative to computation.



4. Conclusion

The MPI-based parallel implementation of SpMV significantly improved performance for larger matrices and a moderate number of processes. Optimal efficiency and speedup were observed between 4 to 12 processes, especially for large matrices. Efficiency declines with higher NP, suggesting communication overhead begins to dominate. Future optimization efforts could focus on reducing MPI communication overhead or implementing smarter data distribution strategies.



. Introduction

Here we evaluate the parallelization performance of a simulation known as "Boids," focusing on comparing three implementations:

- **Sequential (Normal)**
- **OpenMP-based parallelization**
- **Threading Building Blocks (TBB) parallelization**

Benchmarks were conducted to measure the execution time across different CPU core counts to assess scalability and efficiency.

2. Methodology

A bash benchmarking script was implemented to automate the execution of each version of the Boids simulation:

- **Sequential** execution (normal).
- **Parallel execution** using OpenMP and TBB frameworks.

The performance was tested on the following CPU core configurations: 1, 2, 4, 8, 12, 16, and 24 cores. Execution times were captured precisely using timestamps recorded immediately before and after execution.

3. Benchmarking Workflow

- **Sequential Execution:**
 - Run on a single core.
 - Provides baseline execution time.
- **OpenMP Execution:**
 - Set the environment variable `OMP_NUM_THREADS` according to the number of cores.
 - Execute and measure execution time.
- **TBB Execution:**
 - Environment variables `TBB_NUM_THREADS` and `OMP_NUM_THREADS` set according to the number of cores.
 - Execute and measure execution time.

4. Benchmark Results

Cores Normal (s) OpenMP (s) TBB (s)

1	0.251	0.254	0.240
2	-	0.224	0.216
4	-	0.254	0.235
8	-	0.240	0.207
12	-	0.289	0.211
16	-	0.293	0.239
24	-	0.245	0.241

5. Analysis of Results

- **Sequential (Normal):** Execution time serves as a baseline for comparisons, with an execution time of approximately 0.251 seconds.
- **OpenMP:**
 - Shows best performance at 2 cores, after which it fluctuates, peaking at 16 cores (0.293 seconds).
 - Indicates overhead in managing threads at higher core counts, causing decreased performance.
- **TBB:**
 - Demonstrates consistently lower execution times than OpenMP.
 - Achieves the best performance at 8 cores (0.207 seconds).
 - Performance remains relatively stable, showing better scalability and reduced overhead.

6. Implementation Details

The bash script provided systematically:

- Checks if executables (`boids`, `boids_openmp`, `boids_tbb`) exist.
- Runs benchmarks for each scenario, precisely capturing execution time.
- Records results in separate CSV files (`normal.csv`, `openmp.csv`, `tbb.csv`).

This automated approach ensures reproducibility and accurate performance measurement.

7. Conclusion

The benchmarking clearly shows that TBB offers better parallel performance and scalability compared to OpenMP for the Boids simulation. OpenMP experiences overhead at higher core counts, reducing efficiency, while TBB provides stable and superior parallel execution. Future improvements could explore optimizing OpenMP's thread management or further tuning TBB for even greater efficiency.

Appendix:

Benchmark Execution, Log Extraction, and Cleanup

This section details the procedures for running the sparse matrix–vector multiplication (SpMV) benchmarks on our HPC cluster, extracting performance metrics from the generated log files, and cleaning up afterwards.

1. Pre-run Cleanup

Before starting a new test run, remove any existing log files to avoid contamination:

```
rm log-NX*-NPI*
```

Submit the benchmark job using the cluster scheduler. The job script (named `run`) is configured with a 50-minute walltime limit. Note that while the `oarsub` command returns immediately, actual execution may be delayed in the queue.

```
oarsub -S ./run
```

3. Monitoring and Reviewing Output

Monitor your job status using:

```
oarstat -u $USER
```

After job completion, review the output file (e.g., [jobid].out) to verify the job logs:

```
cat [jobid].out
```

4. Log Extraction

Once the benchmark has finished, run the Python extraction script to parse the log files and aggregate performance metrics into a CSV file (e.g., resultats_logs_corriges.csv):

```
bash
Copy
python extraction.py
```

5. Local Testing (Optional)

For quick local testing, you can manually set the number of MPI processes and the matrix dimension. For example, to run the executable with 4 processes and a Laplacian matrix of size 100:

```
NP=4
NX=100
mpirun -np $NP ./bin/sparseemv_mpi2.exe --nx $NX > log-NX${NX}-NPI${NP}
```

6. Post-run Cleanup

After benchmarks and log extractions are complete, remove all generated log files to prepare for subsequent runs:

```
find . -maxdepth 1 -type f -regex './log-.*' -delete
```

Additional Remarks

- **Walltime:** The job script enforces a 50-minute walltime (walltime=00:50:00), which is the maximum allowed per run.
- **Performance:** Higher parallel configurations (e.g., NPI = 8, 12, 16, 24) typically yield significant speedups compared to sequential execution (NPI = 1).
- **Queueing:** Although the `oarsub` command returns immediately, actual job execution may experience queuing delays due to current cluster load.